MODULE 5
OPTIMIZATION TECHNIQUES

Mrs. Jisha Tinsu
Assistant Professor
Dept. of Information Technology, TCET

Contents.....

- Model Selection techniques
- Cross Validation
- Grid Search method
- Gradient descent
- Types of Gradient descent
- Hyper parameter tuning

What Is Model Selection

- Model selection is the process of selecting one final machine learning model from among a collection of candidate machine learning models for a training dataset.
- Model selection is a process that can be applied both across different types of models (e.g. logistic regression, SVM, KNN, etc.) and across models of the same type configured with different model hyperparameters (e.g. different kernels in an SVM).
- For example, we may have a dataset for which we are interested in developing a classification or regression predictive model. We do not know beforehand as to which model will perform best on this problem, as it is unknowable. Therefore, we fit and evaluate a suite of different models on the problem.
- **Model selection** is the process of choosing one of the models as the final model that addresses the problem.

What Is Model Selection contd..

- If we are in a data-rich situation, the best approach is to randomly divide the dataset into three parts:
  - training set,
  - validation set,
  - test set.
- The training set is used to fit the models; the validation set is used to estimate **prediction error** for model selection; the test set is used for assessment of the **generalization error** of the final chosen model.
- Model selection is the process of choosing the best model among a set of candidates, balancing bias and variance for optimal performance.

What Is Model Selection contd..

- **Common Techniques:**
- **Train-Test Split:** Dividing dataset into training and test sets.
- **Cross-Validation (CV):** More reliable, especially for small datasets.
- **Performance Metrics:** Accuracy, Precision, Recall, F1 Score, ROC-AUC, etc.
- **Information Criteria:** AIC (Akaike), BIC (Bayesian) for statistical models.
- **Regularization-based methods:** Lasso, Ridge – add penalty terms to control model complexity.

What Is Model Selection contd..

Three common resampling based model selection methods include:

1. Random split.
2. Bootstrap
3. Cross-Validation (k-fold, LOOCV, etc)

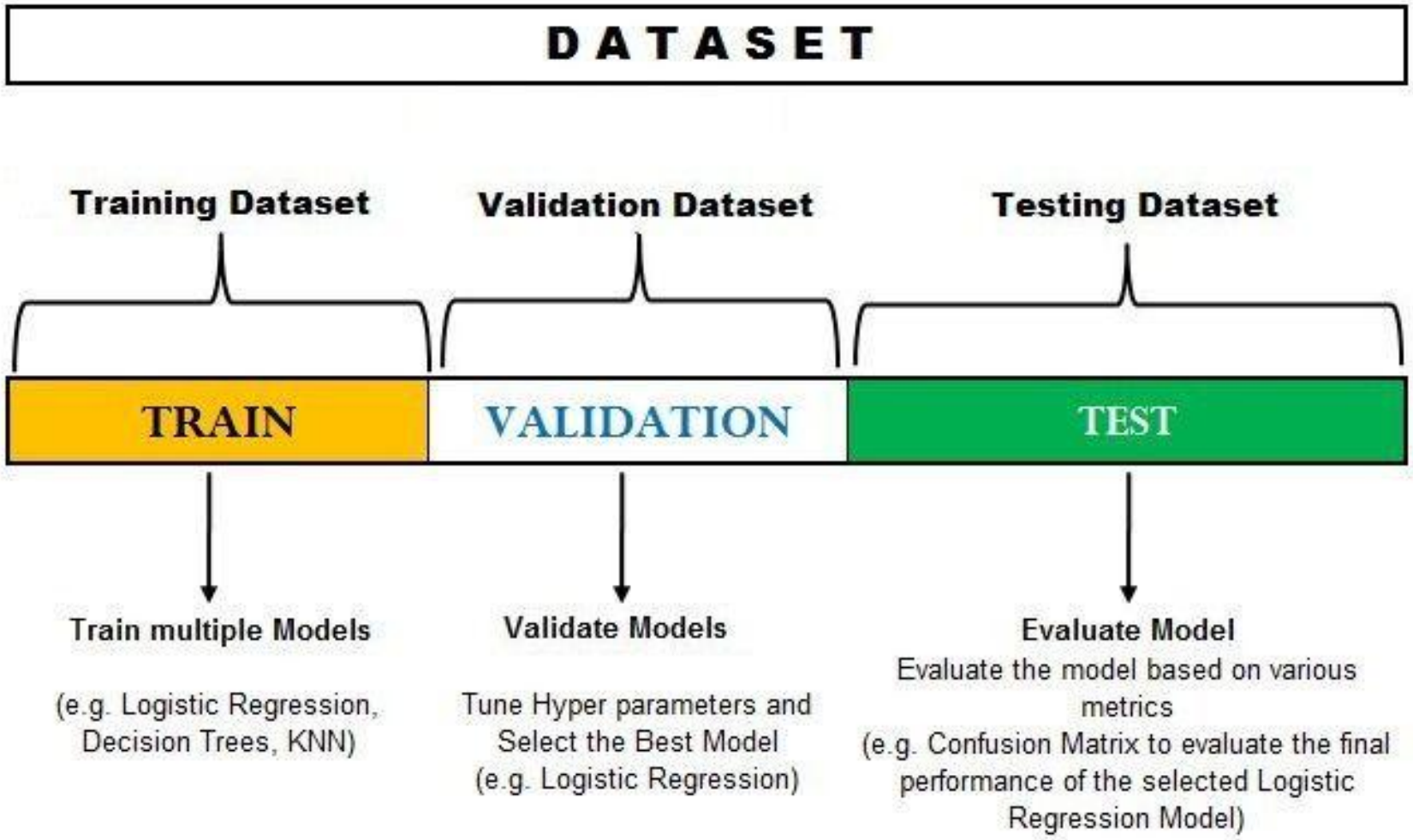
1. Random Split

- Random Splits are used to randomly sample a percentage of data into training, testing, and preferably validation sets.

- The advantage of this method is that there is a good chance that the original population is well represented in all the three sets. In more formal terms, random splitting will prevent a biased sampling of data.

About Train, Validation and Test Sets in Machine Learning

- Training Dataset:
  - The sample of actual data used to fit the model.
- Validation Dataset:
  - The sample of data used to provide an unbiased evaluation of a model fit on the training dataset while tuning model hyperparameters.
  - This dataset helps during the “development” stage of the model.
- Test Dataset:
  - The sample of data used to provide an unbiased evaluation of a final model fit on the training dataset.
  - It is only used once a model is completely trained(using the train and validation sets).


DATASET
Training Dataset
Validation Dataset
Testing Dataset
TRAIN
VALIDATION
TEST
Train multiple Models
(e.g. Logistic Regression, Decision Trees, KNN)
Validate Models
Tune Hyper parameters and Select the Best Model (e.g. Logistic Regression)
Evaluate Model
Evaluate the model based on various metrics
(e.g. Confusion Matrix to evaluate the final performance of the selected Logistic Regression Model)

Time-Based Split

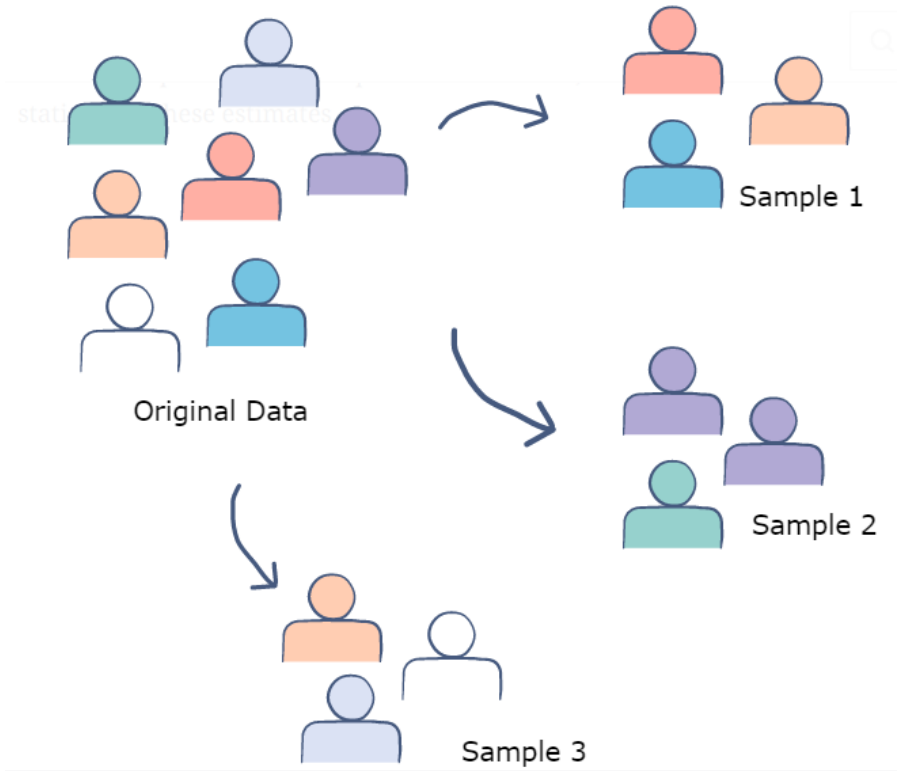
- There are **some types of data where random splits are not possible**.
- For example, if we have to train a model for weather forecasting, we cannot randomly divide the data into training and testing sets.
- This will jumble up the seasonal pattern! Such data is often referred to by the term – Time Series. And In such cases, a time-wise split is used.
- The training set can have data for the last three years and 10 months of the present year. The last two months can be reserved for the testing or validation set.
- There is also a **concept of *window* sets** – where the model is trained till a particular date and tested on the future dates iteratively such that the training window keeps increasing shifting by one day (consequently, the test set also reduces by a day). The advantage of this method is that it stabilizes the model and prevents overfitting when the test set is very small (say, 3 to 7 days).

Holdout Method (one train test split)

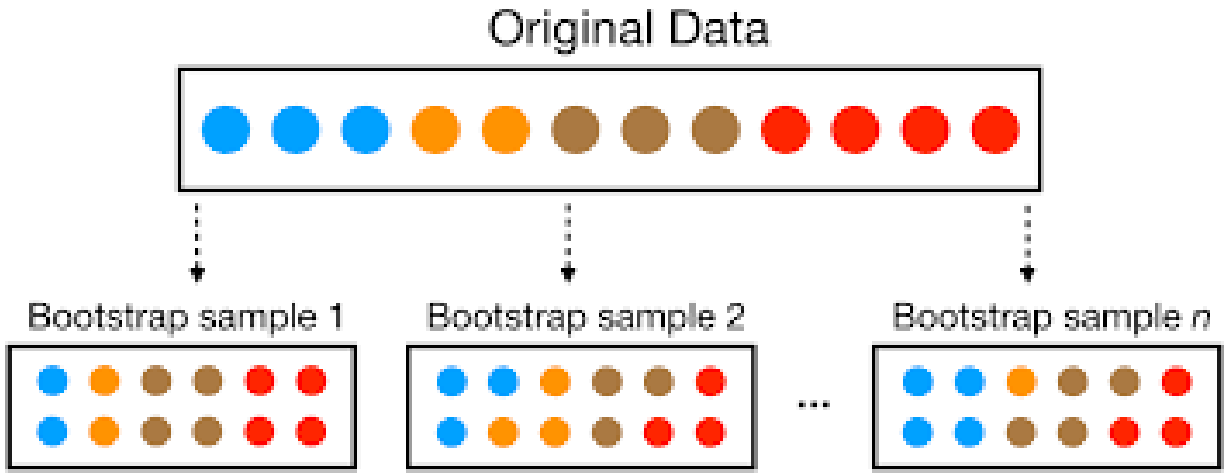
- Hold-out is when you split up your dataset into a ‘train’ and ‘test’ set. The training set is what the model is trained on, and the test set is used to see how well that model performs on unseen data.
- A common split when using the hold-out method is using 80% of data for training and the remaining 20% of the data for testing.
- Hold-out, is dependent on just one train-test split. That makes the hold-out method score dependent on how the data is split into train and test sets.
- Although this approach is simple to perform, it still faces the issue of high variance, and it also produces misleading results sometimes.

2. Bootstrap

- The first step is to select a sample size (which is usually equal to the size of the original dataset). Thereafter, a sample data point must be randomly selected from the original dataset and added to the bootstrap sample.
- After the addition, the sample needs to be put back into the original sample. This process needs to be repeated for N times, where N is the sample size.
- Therefore, it is a resampling technique that creates the bootstrap sample by sampling data points from the original dataset ***with replacement***. This means that the bootstrap sample can contain multiple instances of the same data point.
- The model is trained on the bootstrap sample and then evaluated on all those data points that did not make it to the bootstrapped sample. These are called the ***out-of-bag*** samples.



Bootstrapping



The bootstrap method involves iteratively resampling a dataset with replacement.

3. Cross-Validation

- Cross-validation is a technique for validating the model efficiency by training it on the subset of input data and testing on previously unseen subset of the input data.
- **We can also say that it is a technique to check how a statistical model generalizes to an independent dataset**.
- In machine learning, there is always the need to test the stability of the model. It means based only on the training dataset; we can't fit our model on the training dataset.
- For this purpose, we reserve a particular sample of the dataset, which was not part of the training dataset.
- After that, we test our model on that sample before deployment, and this complete process comes under cross-validation.

Cross-Validation steps

Hence the basic steps of cross-validations are:

1. Reserve a subset of the dataset as a validation set.
2. Provide the training to the model using the training dataset.
3. Now, evaluate model performance using the validation set. If the model performs well with the validation set, perform the further step, else check for the issues.

Methods used for Cross-Validation

a. Validation Set Approach
b. Leave-P-out cross-validation
c. Leave one out cross-validation
d. K-fold cross-validation
e. Stratified k-fold cross-validation

a. Validation Set Approach

- We divide our input dataset into a training set and test or validation set in the validation set approach. Both the subsets are given 50% of the dataset.

- But it has one of the big disadvantages that we are just using a 50% dataset to train our model, so the model may miss out to capture important information of the dataset. It also tends to give the underfitted model.

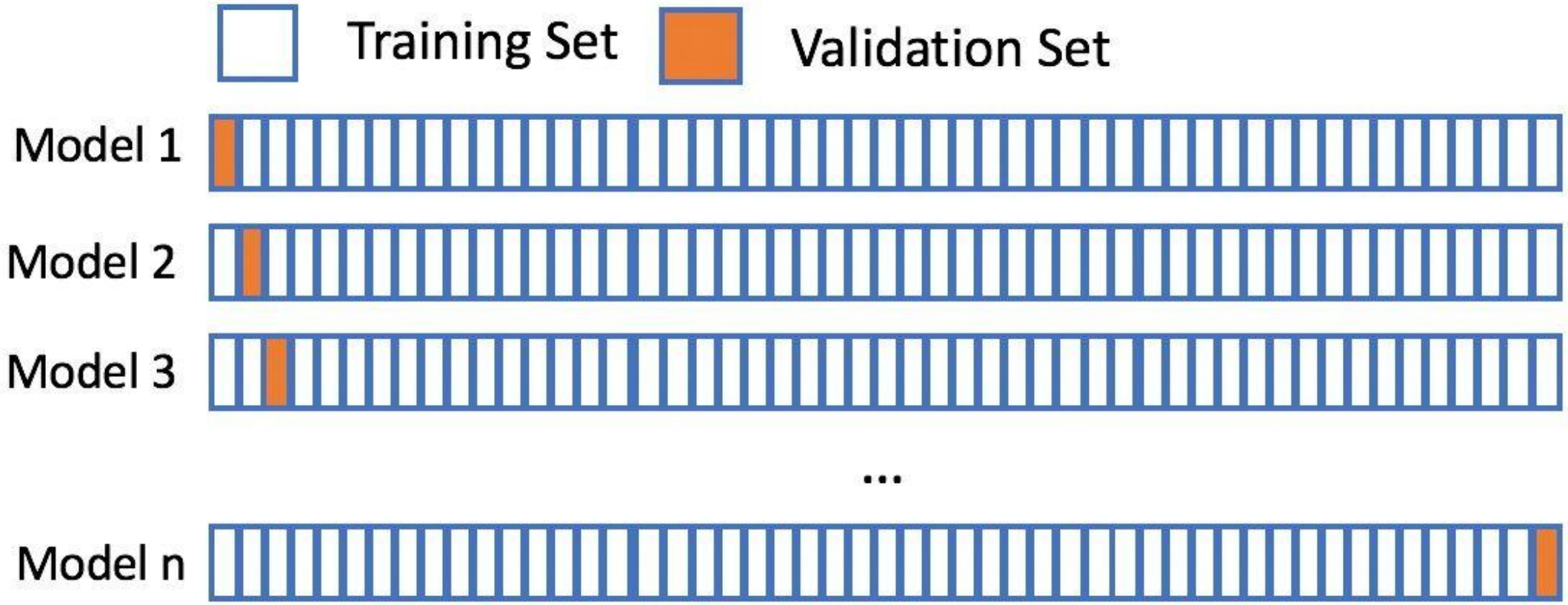
b. Leave-P-out cross-validation

- An exhaustive cross-validation technique, p samples are used as the validation set and n-p samples are used as the training set if a dataset has n samples.
- The process is repeated until the entire dataset containing n samples gets divided on the validation set of p samples and the training set of n-p samples.
- This continues till all samples are used as a validation set.
- The technique, which has a high computation time, produces good results. However, it's not considered ideal for an imbalanced dataset and is deemed to be a computationally unfeasible method.
- This is because if the training set has all samples of one class, the model will not be able to properly generalize and will become biased to either of the classes.

c. Leave one out cross-validation(LOOCV)

- This method is similar to the leave-p-out cross-validation, but instead of p, we need to take 1 dataset out of training.
- It means, in this approach, for each learning set, only one datapoint is reserved, and the remaining dataset is used to train the model.
- This process repeats for each datapoint. Hence for n samples, we get n different training set and n test set.
- It has the following features:

1. In this approach, the bias is minimum as all the data points are used.
2. The process is executed for n times; hence execution time is high.
3. This approach leads to high variation in testing the effectiveness of the model as we iteratively check against one data point.



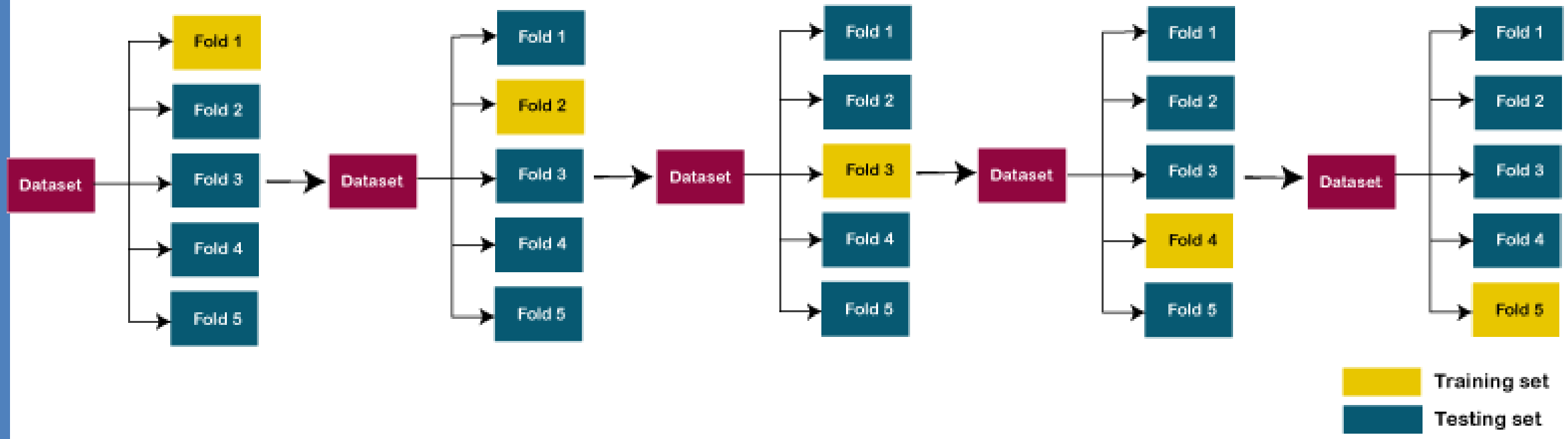
Leave one out cross-validation(LOOCV)

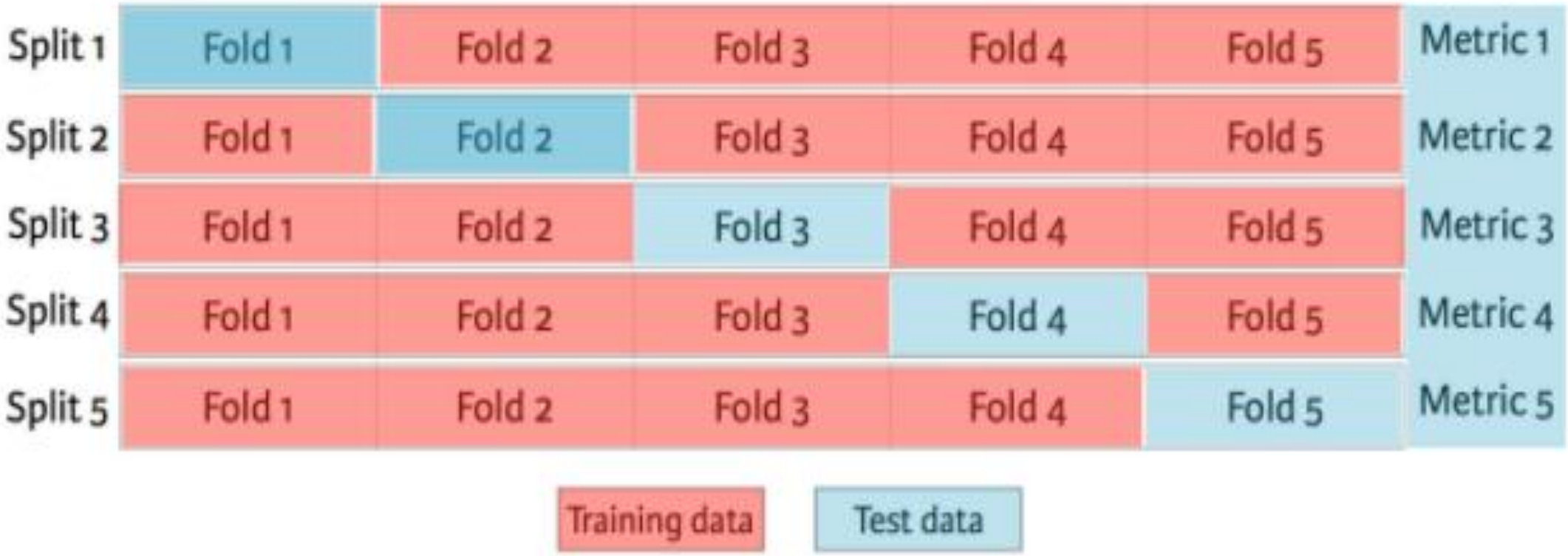
d. K-Fold Cross-Validation

- K-fold cross-validation approach divides the input dataset into K groups of samples of equal sizes. These samples are called **folds**. The steps for k-fold cross-validation are:

1. Split the input dataset into K groups
2. For each group:
   - Take one group as the reserve or test data set.
   - Use remaining groups as the training dataset
   - Fit the model on the training set and evaluate the performance of the model using the test set.

- Example:
- Let's take an example of 5-folds cross-validation. So, the dataset is grouped into 5 folds. On 1st iteration, the first fold is reserved for test the model, and rest are used to train the model. On 2nd iteration, the second fold is used to test the model, and rest are used to train the model. This process will continue until each fold is not used for the test fold.


Dataset
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Dataset
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Dataset
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Dataset
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Dataset
Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Training set
Testing set



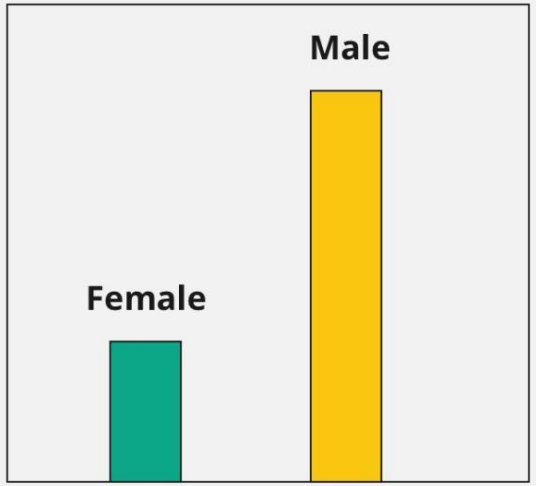
5 fold cross validation

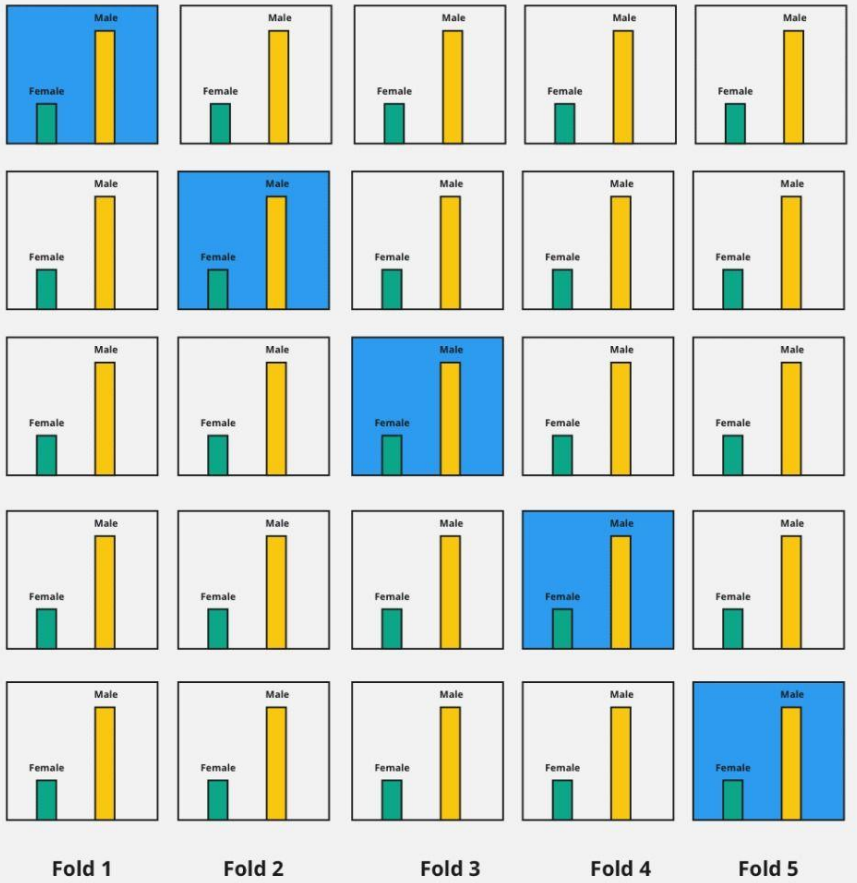
e. Stratified K-Fold

- Stratified k-fold cross-validation is same as just k-fold cross-validation, but in Stratified k-fold cross-validation, it does stratified sampling instead of random sampling.
- If for instance, the target variable is a categorical variable with 2 classes, then stratified k-fold ensures that each test fold gets an equal ratio of the two classes when compared to the training set.
- This makes the model evaluation more accurate and the model training less biased.
- This enables it to work perfectly for imbalanced datasets, but not for time-series data

Stratified K-Fold Cross Validation

Target Class Distribution





dataaspirant.com

Random sampling and Stratified sampling with example

Random sampling

- Suppose you want to take a survey and decided to call 1000 people from a particular state, If you pick either 1000 male completely or 1000 female completely or 900 female and 100 male (randomly) to ask their opinion on a particular product.
- Then based on these 1000 opinion you can't decide the opinion of that entire state on your product. This is random sampling.

Stratified sampling

- In Stratified Sampling, Let the population for that state be 60% male and 40% female, Then for choosing 1000 people from that state if you pick 600 male (60% of 1000) and 400 female (40% for 1000) i.e 600 male + 400 female (Total=1000 people) to ask their opinion.
- Then these groups of people represent the entire state. This is called as Stratified Sampling.

Limitations of Cross-Validation

- For the ideal conditions, it provides the optimum output. But for the inconsistent data, it may produce a drastic result. So, it is one of the big disadvantages of cross-validation, as there is no certainty of the type of data in machine learning.

- In predictive modeling, the data evolves over a period, due to which, it may face the differences between the training set and validation sets. Such as if we create a model for the prediction of stock market values, and the data is trained on the previous 5 years stock values, but the realistic future values for the next 5 years may drastically different, so it is difficult to expect the correct output for such situations.

Hyper parameter tuning

- A Machine Learning model is defined as a mathematical model with a number of parameters that need to be learned from the data. By training a model with existing data, we are able to fit the model parameters.
- However, there is another kind of parameters, known as ***Hyperparameters***, that cannot be directly learned from the regular training process.
- These parameters express important properties of the model such as its complexity or how fast it should learn

Some examples of model hyperparameters include:

- The k in k-nearest neighbors.
- Number of layers in neural network
- The penalty in Logistic Regression Classifier i.e. L1 or L2 regularization
- The learning rate for training a neural network.
- The C and sigma (Gamma) hyperparameters for support vector machines.

Strategies for Hyperparameter tuning

Models can have many hyperparameters and finding the best combination of parameters can be treated as a search problem.

Best strategies for Hyperparameter tuning based on **Grid searching** are:

1 GridSearchCV

2 RandomizedSearchCV

Gradient-Based Optimization are-

1. Gradient Descent
2. Stochastic Gradient Descent (SGD)
3. Batch Gradient Descent
4. Mini Batch gradient descent

Grid Searching

- Grid searching is a method to find the best possible combination of hyper-parameters at which the model achieves the highest accuracy.
- Before applying Grid Searching on any algorithm, Data is used to divided into training and validation set, a validation set is used to validate the models. A model with all possible combinations of hyperparameters is tested on the validation set to choose the best combination.
- For example, we can apply grid searching on K-Nearest Neighbors by validating its performance on a set of values of K in it. Same thing we can do with Logistic Regression by using a set of values of learning rate to find the best learning rate at which Logistic Regression achieves the best accuracy.

GridSearchCV

- In GridSearchCV approach, machine learning model is evaluated for a range of hyperparameter values.
- This approach is called GridSearchCV because it searches for best set of hyperparameters from a grid of hyperparameters values
- For example, if we want to set two hyperparameters C and Alpha of Logistic Regression Classifier model, with different set of values.
- The gridsearch technique will construct many versions of the model with all possible combinations of hyerparameters, and will return the best one.

GridSearchCV example

Consider the example of logistic regression where, two hyperparameters C and alpha have to be set for optimized model.

Let us have, C = [0.1, 0.2, 0.3, 0.4, 0.5] and Alpha = [0.1, 0.2, 0.3, 0.4]. For a combination ***C=0.3 and Alpha=0.2*** as shown in below image, performance score comes out to be **0.726(Highest)**, therefore it is

C \ Alpha	0.1	0.2	0.3	0.4
0.5	0.701	0.703	0.697	0.696
0.4	0.699	0.702	0.698	0.702
0.3	0.721	0.726	0.713	0.703
0.2	0.706	0.705	0.704	0.701
0.1	0.698	0.692	0.688	0.675

Drawback of GridSearchCV :

GridSearchCV will go through all the intermediate combinations of hyperparameters which makes grid search computationally very expensive

RandomizedSearchCV

- RandomizedSearchCV solves the drawbacks of GridSearchCV, as it goes through only a fixed number of hyperparameter settings.

- It moves within the grid in random fashion to find the best set hyperparameters. This approach reduces unnecessary computation.

Gradient-Based Optimization

- Gradient-based optimization is a fundamental technique used to minimize or maximize functions iteratively.
- In the context of machine learning, it's commonly used to minimize a loss function with respect to the model's parameters.

Gradient-Based Optimization -

1. Gradient Descent
2. Stochastic Gradient Descent (SGD)
3. Batch Gradient Descent
4. Mini Batch gradient descent

1. Gradient Descent algorithm

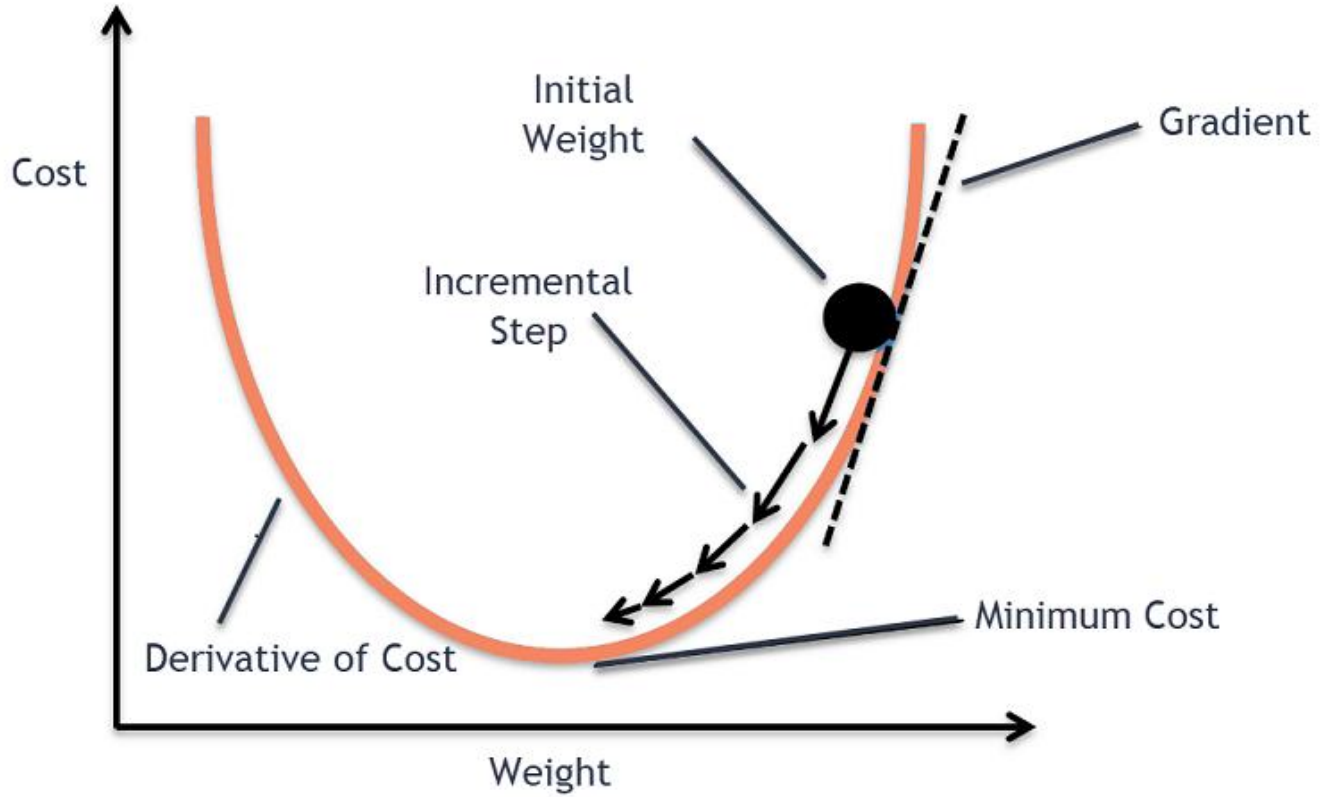
- Gradient Descent is an optimization algorithm used for minimizing the cost function in various machine learning algorithms.
- It is basically used for updating the parameters of the learning model.

What is a Cost Function?

- It is a **function** that measures the performance of a model for any given data.
- **Cost Function** quantifies the error between predicted values and expected values and presents it in the form of a single real number.

Gradient Descent algorithm contd..

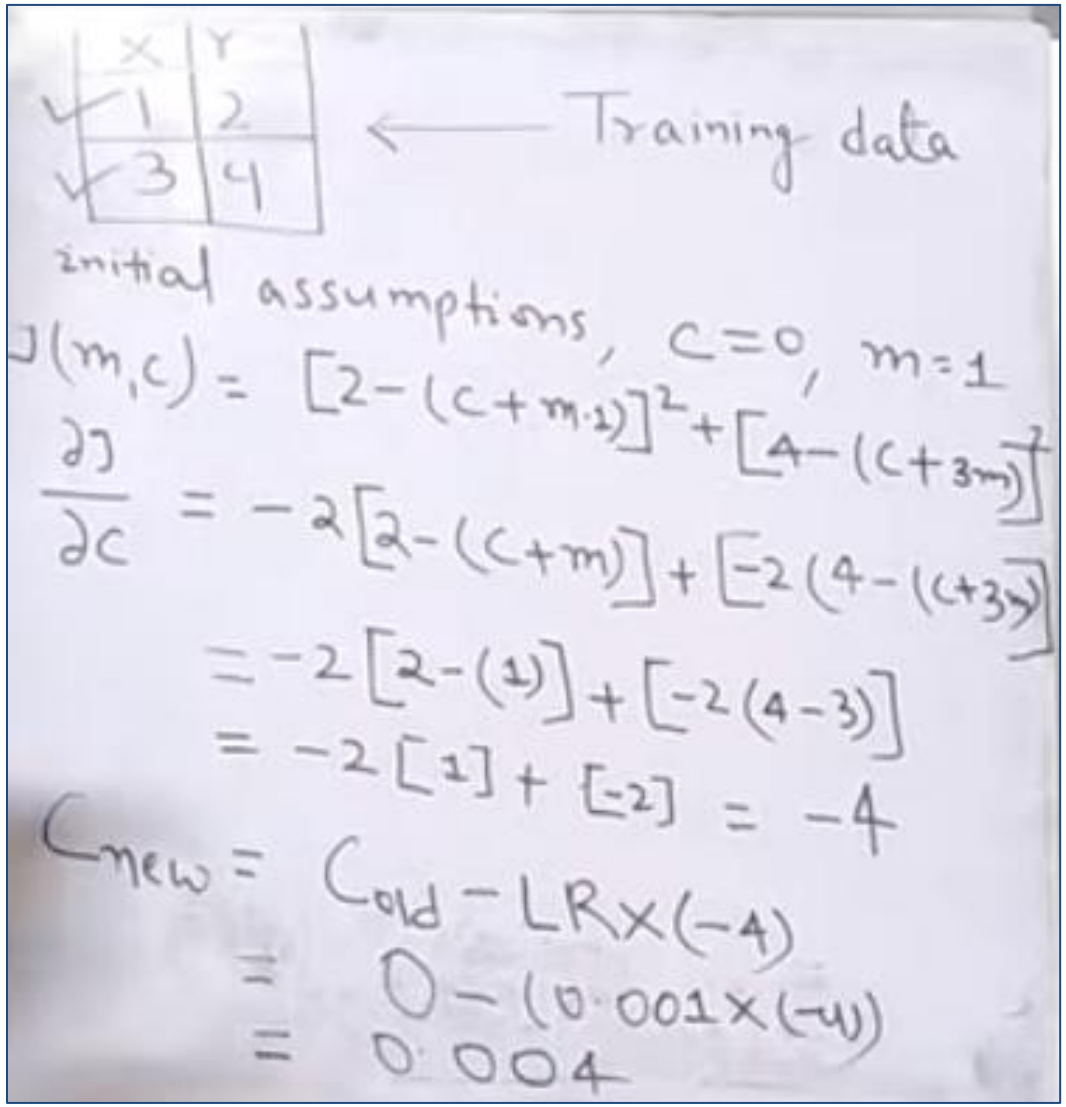
- Gradient descent is an iterative optimization algorithm for finding the local minimum of a function.
- To find the local minimum of a function using gradient descent, we must take steps proportional to the negative of the gradient (move away from the gradient) of the function at the current point.
- If we take steps proportional to the positive of the gradient (moving towards the gradient), we will approach a local maximum of the function, and the procedure is called **Gradient Ascent.**
- The gradient is the vector containing all partial derivatives of a function in a point
- We can apply gradient descent on a convex function, and gradient ascent on a concave function
- Gradient descent finds the nearest minimum of a function, gradient ascent the nearest maximum



The goal of the gradient descent algorithm is to minimize the given function (say cost function). To achieve this goal, it performs two steps iteratively:

1. **Compute the gradient** (slope), the first order derivative of the function at that point
2. **Make a step (move) in the direction opposite to the gradient**, opposite direction of slope increase from the current point by alpha times the gradient at that point

Example: Linear Regression



Consider an example, where we have to optimized the values of c and m in linear regression formula y=mx + c

New value = old value - step size
=old value - (learning rate * slope)

Where, learning rate→ how aggressively you want to change step
Slope→ change in y coordinate w.r.t. change in x coordinate

Cost function for linear regression

$$J(m,c) = \sum (Y_i - (mX_i + c)^2$$

For $\sum$ i=1 to n

Gradient descent algorithm terms involved

Hypothesis: $h_\theta(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}}\ J(\theta_0, \theta_1)$

Gradient descent algorithm

$$\text{repeat until convergence } \{$$

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

$$(\text{for } j = 1 \text{ and } j = 0)$$

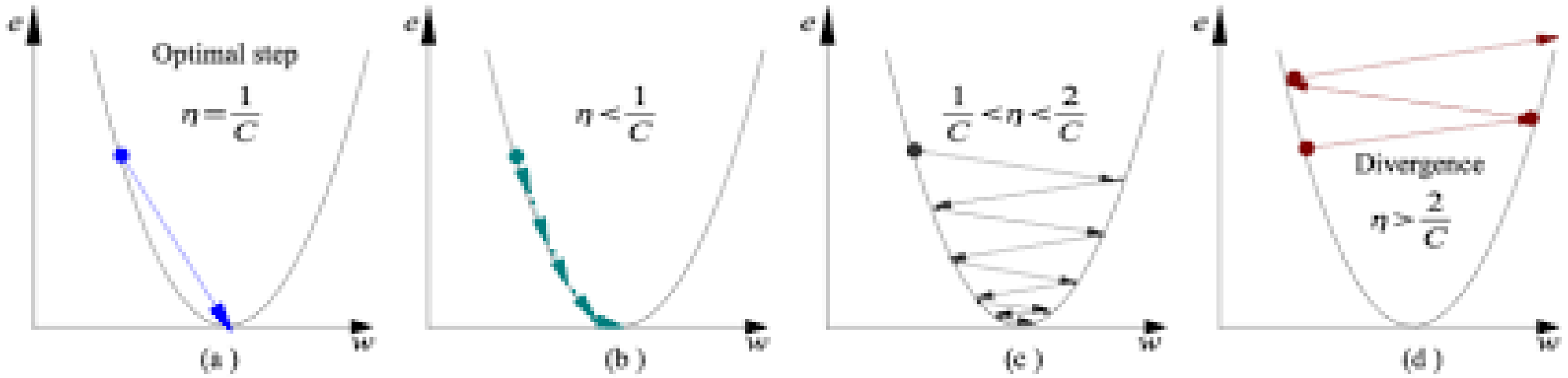
$$\}$$

Alpha is called **Learning rate** – a tuning parameter in the optimization process. It decides the length of the steps

Alpha – The Learning Rate

- ✓ We have the direction we want to move in, now we must decide the size of the step we must take.
- ✓ It must be chosen carefully to end up with local minima.
- ✓ If the learning rate is too high, we might OVERSHOOT the minima and keep bouncing, without reaching the minima
- ✓ If the learning rate is too small, the training might turn out to be too long

Alpha – The Learning Rate cases



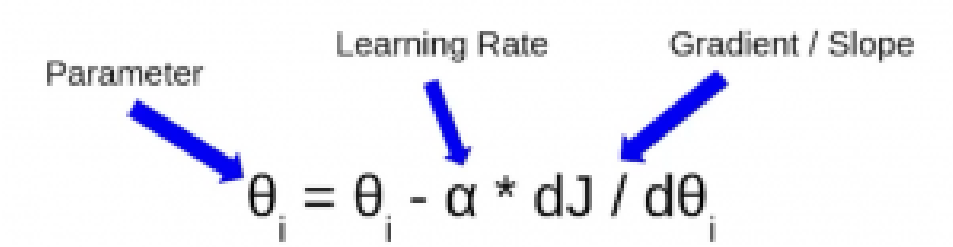
a) **Case 1:** Learning rate is optimal, model converges to the minimum

b) **Case 2:** Learning rate is too small, it takes more time but converges to the minimum

c) **Case 3:** Learning rate is higher than the optimal value, it overshoots but converges ($1/C < \eta < 2/C$)

d) **Case 4:** Learning rate is very large, it overshoots and diverges, moves away from the minima, performance decreases on learning

What is the equation of the Gradient Descent Algorithm?



Here,

- **θ** is the parameter we wish to update,
- **dJ/dθ** is the partial derivative which tells us the rate of change of error on the cost function with respect to the parameter θ,
- **α** here is the Learning Rate.
- So, this **J** here represents the cost function and there are multiple ways to calculate this cost. Based on the way we are calculating this cost function there are different variants of Gradient Descent.

Variants of gradient Descent:

1. Batch Gradient Descent:

- This is a type of gradient descent which processes all the training examples for each iteration of gradient descent.
- But if the number of training examples is large, then batch gradient descent is computationally very expensive.
- Hence if the number of training examples is large, then batch gradient descent is not preferred.
- Instead, we prefer to use stochastic gradient descent or mini-batch gradient descent.

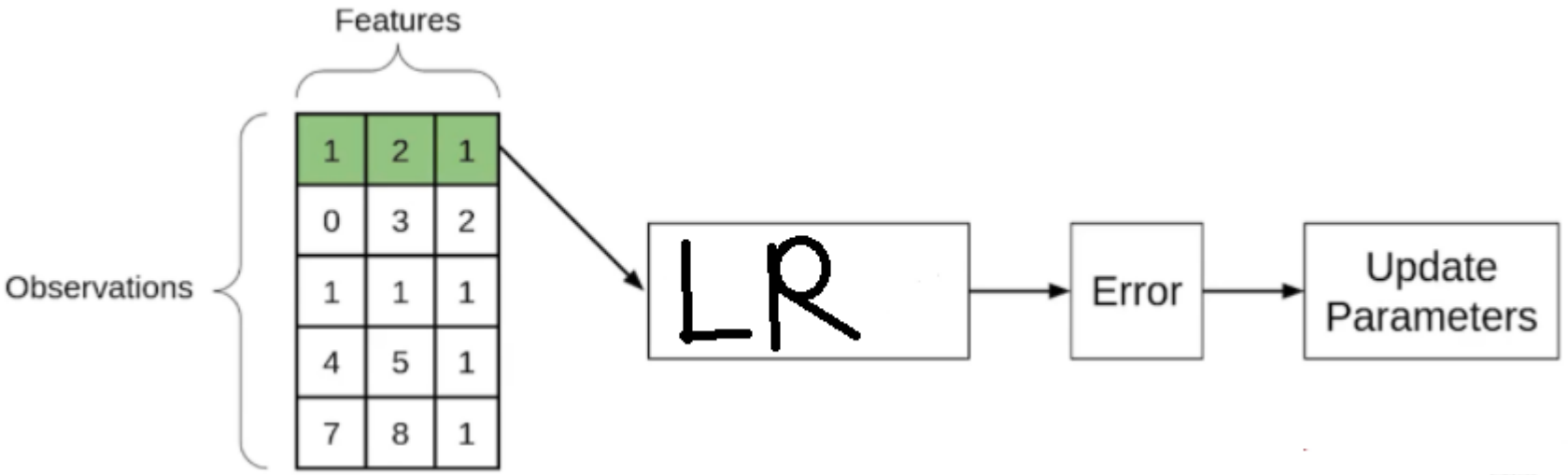
Batch Gradient Descent contd..

- If there are a total of ‘m’ observations in a data set then we use all these observations to calculate the cost function **J**, then this is known as **Batch Gradient Descent**.
- So for the entire training set, we calculate the cost function. And then we update the parameters using the rate of change of this cost function with respect to the parameters.
- An **epoch** is when the entire training set is passed through the model.
- In batch Gradient Descent since we are using the entire training set, the parameters will be updated only once per epoch.

2. Stochastic Gradient Descent

- If you use a single observation to calculate the cost function it is known as **Stochastic Gradient Descent,** commonly abbreviated as **SGD**. We pass a single observation at a time, calculate the cost and update the parameters.
- This is a type of gradient descent which processes 1 training example per iteration.
- Hence, the parameters are being updated even after one iteration in which only a single example has been processed.
- Hence this is quite faster than batch gradient descent.
- But again, when the number of training examples is large, even then it processes only one example which can be additional overhead for the system as the number of iterations will be quite large.

- if we use the SGD, will take the first observation, then pass it through the neural network, calculate the error and then update the parameters.
- Then will take the second observation and perform similar steps with it.
- This step will be repeated until all observations have been passed through the network and the parameters have been updated.
- Each time the parameter is updated, it is known as an Iteration.
- Here since we have 5 observations, the parameters will be updated 5 times or we can say that there will be 5 iterations.

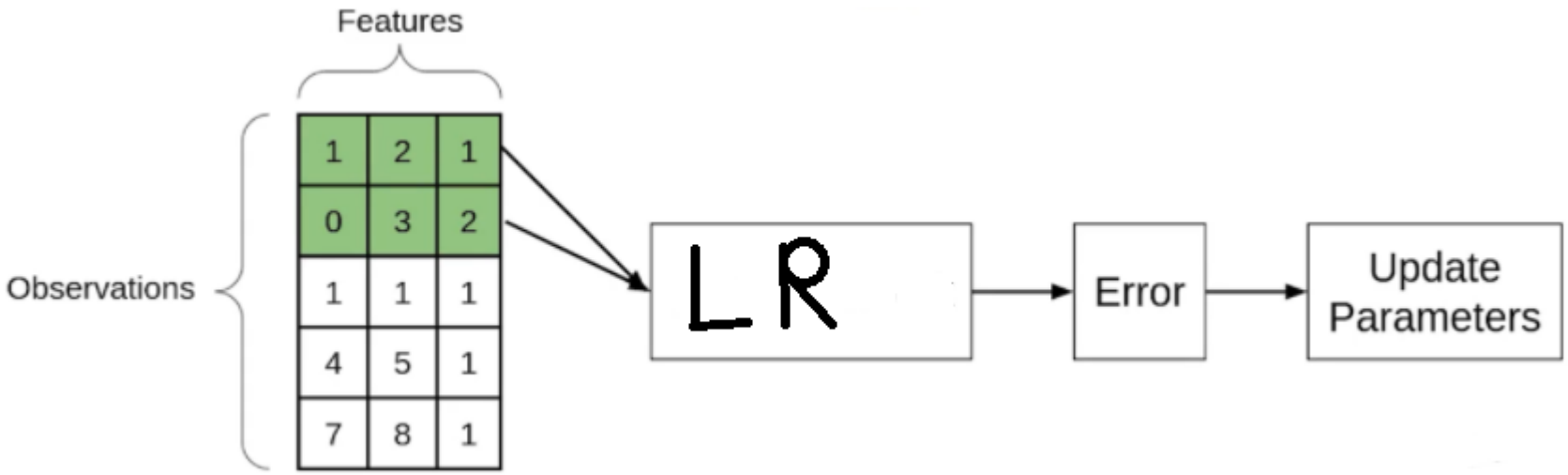


Stochastic Gradient Descent

3. Mini Batch gradient descent

- It takes a subset of the entire dataset to calculate the cost function. So if there are 'm' observations then the number of observations in each subset or mini-batches will be more than 1 and less than 'm'.
- This is a type of gradient descent which works faster than both batch gradient descent and stochastic gradient descent.
- Here b examples where $b<m$ are processed per iteration.
- So even if the number of training examples is large, it is processed in batches of b training examples in one go.
- Thus, it works for larger training examples and that too with lesser number of iterations.

- Assume that the batch size is 2. So we'll take the first two observations, pass them linear regression model, calculate the error and then update the parameters.
- Then we will take the next two observations and perform similar steps i.e will pass through the network, calculate the error and update the parameters.



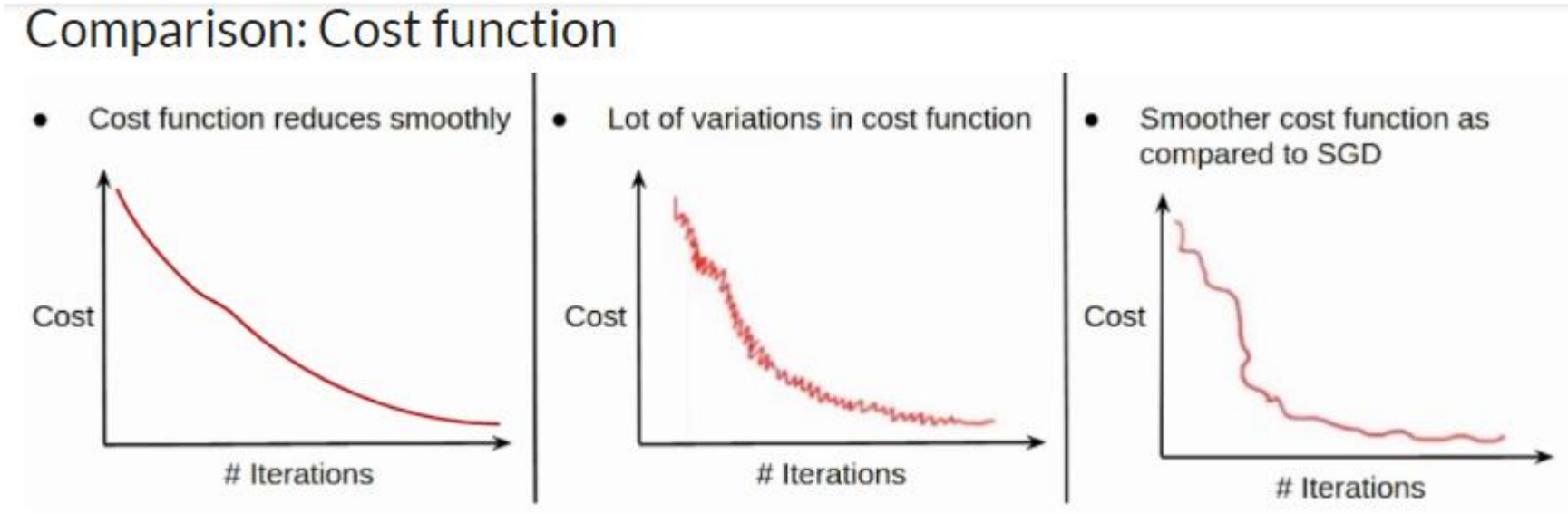
Mini Batch gradient descent

Comparison between batch, stochastic and mini batch gradient descent

•In batch gradient Descent, as we have seen earlier as well, we take the entire dataset > calculate the cost function > update parameter.

•In the case of Stochastic Gradient Descent, we update the parameters after every single observation and we know that every time the weights are updated it is known as an iteration.

•In the case of Mini-batch Gradient Descent, we take a subset of data and update the parameters based on every subset.



•Now since we update the parameters using the entire data set in the case of the Batch GD, the cost function, in this case, reduces smoothly.

•On the other hand, this updation in the case of SGD is not that smooth. Since we're updating the parameters based on a single observation, there are a lot of iterations. It might also be possible that the model starts learning noise as well.

•The updation of the cost function in the case of Mini-batch Gradient Descent is smoother as compared to that of the cost function in SGD. Since we're not updating the parameters after every single observation but after every subset of the data.

Comparison: Computation Cost and Time

• Computation cost is very high	• Computation time is more	• Computation time is lesser than SGD • Computation cost is lesser than Batch Gradient Descent

•Since we've to load the entire data set at a time, perform the forward propagation on that and calculate the error and then update the parameters, the computation cost in the case of Batch gradient descent is very high.

•Computation cost in the case of SGD is less as compared to the Batch Gradient Descent since we've to load every single observation at a time but the Computation time here increases as there will be more number of updates which will result in more number of iterations.

•In the case of Mini-batch Gradient Descent, taking a subset of the data there are a lesser number of iterations or updations and hence the computation time in the case of mini-batch gradient descent is less than SGD. Also, since we're not loading the entire dataset at a time whereas loading a subset of the data, the computation cost is also less as compared to the Batch gradient descent

Comparison between Batch GD, SGD, and Mini-batch GD:

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
• Entire dataset for updation	• Single observation for updation	• Subset of data for updation
• Cost function reduces smoothly	• Lot of variations in cost function	• Smoother cost function as compared to SGD
• Computation cost is very high	• Computation time is more	• Computation time is lesser than SGD
		• Computation cost is lesser than Batch Gradient Descent

General terms in machine learning optimization techniques

What Is a Sample?

- A sample is a single row of data.
- It contains inputs that are fed into the algorithm and an output that is used to compare to the prediction and calculate an error.
- A training dataset is comprised of many rows of data, e.g. many samples. A sample may also be called an instance, an observation, an input vector, or a feature vector.

What Is a Batch?

- The batch size is a hyperparameter that defines the number of samples to work through before updating the internal model parameters.

General terms in machine learning optimization techniques contd..

- A training dataset can be divided into one or more batches.
- When all training samples are used to create one batch, the learning algorithm is called batch gradient descent. When the batch is the size of one sample, the learning algorithm is called stochastic gradient descent. When the batch size is more than one sample and less than the size of the training dataset, the learning algorithm is called mini-batch gradient descent.
- **Batch Gradient Descent**. Batch Size = Size of Training Set
- **Stochastic Gradient Descent**. Batch Size = 1
- **Mini-Batch Gradient Descent**. 1 < Batch Size < Size of Training Set
- In the case of mini-batch gradient descent, popular batch sizes include 32, 64, and 128 samples.

General terms in machine learning optimization techniques
contd..

What Is an Epoch?

- The number of epochs is a hyperparameter that defines the number times that the learning algorithm will work through the entire training dataset.
- One epoch means that each sample in the training dataset has had an opportunity to update the internal model parameters. An epoch is comprised of one or more batches. For example, as above, an epoch that has one batch is called the batch gradient descent learning algorithm.

General terms in machine learning optimization techniques
contd..

What Is the Difference Between Batch and Epoch?

- The batch size is a number of samples processed before the model is updated.
- The number of epochs is the number of complete passes through the training dataset.
- The size of a batch must be more than or equal to one and less than or equal to the number of samples in the training dataset.

General terms in machine learning optimization techniques: Example

- Assume you have a dataset with 200 samples (rows of data) and you choose a batch size of 5 and 1,000 epochs.
- This means that the dataset will be divided into 40 batches, each with five samples. The model weights will be updated after each batch of five samples.
- This also means that one epoch will involve 40 batches or 40 updates to the model.
- With 1,000 epochs, the model will be exposed to or pass through the whole dataset 1,000 times. That is a total of 40,000 batches during the entire training process.

- **Gradient Descent algorithm**

https://www.youtube.com/watch?v=vsWrXfO3wWw

Thank you